

Universidad Nacional de Ingeniería

Facultad de Ciencias

Escuela de Ciencia de la Computación

Estudio Comparativo de Estrategias de Entrenamiento en Redes Neuronales Convolucionales



Curso: Inteligencia Artificial CC421A

Docente: Alania Vicente, Marcos Antonio

Integrantes:

- | | |
|--------------------------------------|-----------|
| ■ Chowdhury Gomez, Junal Johir | 20200092K |
| ■ La Torre Vasquez, Andres Sebastian | 20212100C |
| ■ Mora Evangelista, Fernando Fausto | 20221429D |
| ■ Osorio-Tello-Jesus Diego | 20211372J |
| ■ Zapata Inga, Janio Adolfo | 20212636K |

8 de diciembre de 2025

Resumen

El presente estudio aborda uno de los desafíos más críticos en el campo de la visión artificial: la dependencia de grandes volúmenes de datos etiquetados para el entrenamiento eficaz de Redes Neuronales Convolucionales (CNN). Se diseñó un experimento comparativo para evaluar la viabilidad del *Transfer Learning* como mecanismo de mitigación ante la escasez de datos, utilizando un dominio de origen numérico (dataset MNIST) aplicado a un dominio objetivo geométrico. Para ello, se construyó un dataset original de 2027 imágenes distribuidas en 8 clases de formas geométricas. La metodología contrastó tres escenarios experimentales:

- (1) Un modelo entrenado desde cero bajo condiciones de escasez (30 % de los datos)
- (2) Un modelo pre-entrenado con MNIST y ajustado al dataset completo (Transfer Learning),
- y (3) Un modelo entrenado desde cero con abundancia de datos (100 % del dataset).

Los resultados experimentales validaron la hipótesis planteada, revelando un rendimiento escalonado. El modelo bajo condiciones de escasez alcanzó una exactitud del **81.98 %**, evidenciando dificultades de generalización. La aplicación de Transfer Learning elevó el rendimiento al **85.28 %**, demostrando que la extracción de características de bajo nivel (bordes y curvas) es transferible entre dominios dispares. Finalmente, el entrenamiento supervisado con la totalidad de los datos nativos estableció el límite superior de rendimiento con un **89.09 %**. Este estudio concluye que, si bien la especialización con datos nativos es superior, el Transfer Learning constituye una estrategia robusta y eficiente para recuperar capacidad predictiva en escenarios donde la recolección de datos es costosa o limitada.

Palabras clave: Redes Neuronales Convolucionales, Transfer Learning, MNIST, Escasez de Datos, Clasificación de Imágenes.

1. Introducción

1.1. Contexto

La visión artificial ha experimentado una revolución sin precedentes en la última década, impulsada principalmente por el desarrollo de arquitecturas de Aprendizaje Profundo (*Deep Learning*) y, específicamente, las Redes Neuronales Convolucionales (CNN). Estos modelos han superado el desempeño humano en tareas complejas de reconocimiento de patrones, desde el diagnóstico médico hasta la conducción autónoma. Sin embargo, este éxito tiene un costo operativo significativo: la necesidad imperiosa de grandes volúmenes de datos etiquetados de alta calidad. En entornos académicos o industriales con recursos limitados, la adquisición y etiquetado manual de miles de imágenes representa un cuello de botella que a menudo inviabiliza el desarrollo de soluciones robustas.

1.2. Planteamiento del Problema

El dilema central que motiva esta investigación surge cuando nos enfrentamos a la “escasez de datos”. Una CNN entrenada con pocas muestras tiende a memorizar los ejemplos de entrenamiento (*overfitting*) en lugar de aprender características generalizables, resultando en un desempeño pobre ante datos nuevos. Ante esta problemática, surge la interrogante sobre la eficacia de las técnicas de *Transfer Learning* (Transferencia de Aprendizaje) [1]. Tradicionalmente, se asume que la transferencia es efectiva cuando los dominios de origen y destino son similares (ej. fotos de perros a fotos de gatos). Sin embargo, ¿es posible transferir conocimiento de un dominio abstracto y numérico, como los dígitos manuscritos del dataset MNIST, hacia un problema de clasificación de formas geométricas puras? Si los filtros de las primeras capas de una CNN aprenden a detectar bordes y curvas simples, teóricamente, este conocimiento debería ser reutilizable independientemente del contexto semántico de la imagen.

1.3. Objetivos del Proyecto

El objetivo general de este trabajo es evaluar cuantitativamente el impacto de diferentes estrategias de entrenamiento en el desempeño de un clasificador de formas geométricas, determinando si el conocimiento previo puede compensar la falta de datos nativos.

Para lograr esto, se definen los siguientes objetivos específicos:

- Construir y curar un dataset propio (“Mi_Dataset”) de 8 clases de figuras geométricas utilizando una herramienta de dibujo digital desarrollada para este fin.
- Implementar tres escenarios experimentales controlados: Escasez de Datos (Red C), Transferencia de Aprendizaje desde MNIST (Red A) y Abundancia de Datos (Red B).
- Analizar el comportamiento de las métricas de *Accuracy* y *Loss* para determinar la convergencia y capacidad de generalización de cada modelo.
- Comparar las matrices de confusión para identificar patrones de error específicos y cómo el pre-entrenamiento influye en la detección de geometrías complejas.

1.4. Hipótesis de Investigación

Basándonos en la teoría del aprendizaje jerárquico de las redes convolucionales, se postula la siguiente relación de desempeño entre los modelos:

$$Rendimiento(C) < Rendimiento(A) < Rendimiento(B) \quad (1)$$

Donde se espera que:

1. La **Red C** (entrenada desde cero con solo el 30 % de datos) presente el desempeño más bajo debido a la falta de variabilidad en las muestras de entrenamiento.
2. La **Red A** (Transfer Learning desde MNIST) supere a la Red C, dado que los pesos pre-entrenados proporcionan una base sólida para la detección de características espaciales básicas, acelerando la convergencia y mejorando la generalización.
3. La **Red B** (entrenada desde cero con el 100 % de datos nativos) obtenga el mejor desempeño global, ya que sus filtros se optimizarán exclusivamente para las características específicas del dataset objetivo sin el sesgo del dominio de origen.

2. Marco Teórico

2.1. Redes Neuronales Convolucionales (CNN)

Las Redes Neuronales Convolucionales (CNN) constituyen la arquitectura dominante en el procesamiento de imágenes debido a su capacidad para aprender representaciones jerárquicas de los datos. A diferencia de las redes neuronales densas tradicionales, las CNN preservan la estructura espacial de la imagen mediante operaciones de convolución [2]. Una CNN típica se compone de dos bloques funcionales:

- **Extractor de Características (Feature Extractor):** Compuesto por capas *Conv2D* que aplican filtros (kernels) para detectar patrones visuales como bordes, texturas y formas simples, seguidas de capas de *Pooling* (como *MaxPooling2D*) que reducen la dimensionalidad espacial, conservando la información más relevante y proporcionando invarianza a pequeñas traslaciones.
- **Clasificador (Top):** Generalmente formado por capas densas (*Dense*) que toman el vector de características aplanado (*Flatten*) y realizan la clasificación final mediante una función de activación no lineal, típicamente *Softmax* para problemas multiclase.

2.2. Estrategias de Entrenamiento

2.2.1. Entrenamiento Supervisado (From Scratch)

Consiste en inicializar los pesos de la red con valores aleatorios y ajustarlos iterativamente mediante el algoritmo de retropropagación (*Backpropagation*) utilizando exclusivamente el dataset objetivo. Esta estrategia requiere una gran cantidad de datos para evitar el sobreajuste (*Overfitting*) [3], donde la red memoriza el ruido de los datos de entrenamiento en lugar de aprender características generalizables.

2.2.2. Transferencia de Aprendizaje (Transfer Learning)

El *Transfer Learning* es una técnica que permite reutilizar el conocimiento adquirido por un modelo en una tarea origen (Dominio Fuente D_S) para mejorar el aprendizaje en una tarea destino (Dominio Objetivo D_T). La hipótesis subyacente es que las primeras capas de una CNN aprenden características de bajo nivel (bordes, curvas, esquinas) que son universales para casi cualquier problema visual [4]. En este estudio, utilizamos el dataset MNIST (dígitos) como D_S . Aunque los dominios semánticos difieren (números vs. formas geométricas), estructuralmente comparten patrones básicos de trazo y curvatura, lo que justifica la transferencia.

2.3. Métricas de Evaluación

Para cuantificar el desempeño de los modelos, se utilizan las siguientes métricas derivadas de la matriz de confusión:

- **Accuracy (Exactitud):** Proporción total de predicciones correctas.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (2)$$

- **Precision (Precisión):** Capacidad del modelo para no etiquetar como positivo una muestra que es negativa.
- **Recall (Sensibilidad):** Capacidad del modelo para encontrar todas las muestras positivas.
- **F1-Score:** Media armónica entre Precisión y Recall, ideal para evaluar el equilibrio del modelo, especialmente si existen desbalances leves entre clases.

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \quad (3)$$

3. Metodología

3.1. Adquisición y Construcción del Dataset

Para garantizar la originalidad y el control sobre las variables del experimento, se optó por la construcción de un dataset propio denominado “Mi_Dataset”.

3.1.1. Herramienta de Recolección

Se desarrolló una aplicación web personalizada (Single Page Application) desplegada en la plataforma Netlify (<https://dibujos-figuras-entrenamiento.netlify.app/>). Esta herramienta permite al usuario dibujar formas geométricas en un lienzo digital (*canvas*) y descargarlas automáticamente con el formato y resolución requeridos para el entrenamiento.

3.1.2. Características del Dataset Objetivo

Las imágenes recolectadas fueron almacenadas y versionadas en un repositorio público de GitHub (https://github.com/JunalChowdhuryG/Mi_Dataset).

- **Volumen Total:** 2027 imágenes.
- **Clases (8):** Círculo, Cuadrado, Equis, Estrella, Rectángulo, Rombo, Semicírculo, Triángulo.
- **Formato:** Escala de grises, redimensionadas a 28×28 píxeles para ser compatibles con la arquitectura de entrada estándar de modelos tipo LeNet/MNIST.
- **Preprocesamiento:** Normalización de valores de píxeles al rango $[0, 1]$ dividiendo por 255.0.

3.1.3. Muestra del Dataset Objetivo

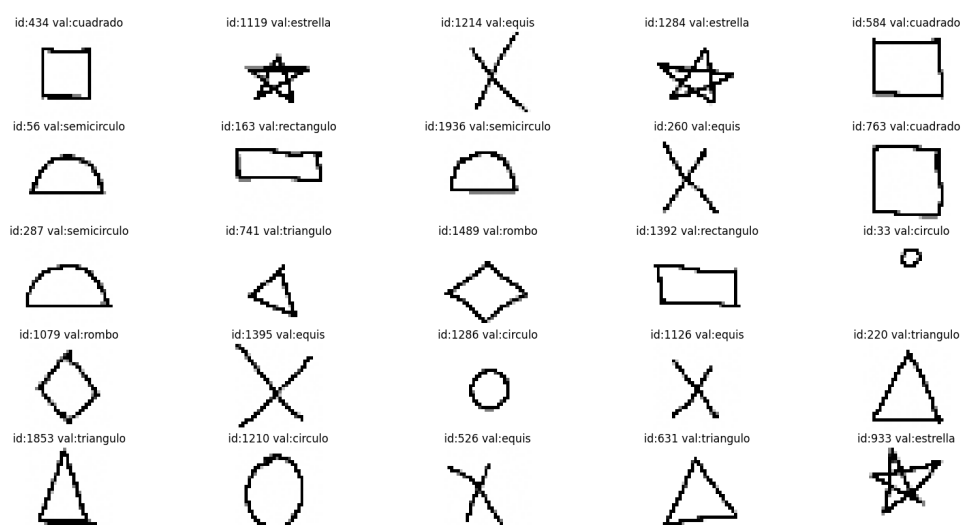


Figura 1: Muestra del Dataset de figuras recolectado

3.2. Dataset Fuente (MNIST)

Como base para la estrategia de transferencia, se utilizó el dataset público MNIST [2], que contiene 60,000 imágenes de entrenamiento y 10,000 de prueba de dígitos manuscritos (0 – 9). Este dataset fue seleccionado debido a su simplicidad visual, que obliga a la red a aprender filtros de detección de trazos muy similares a los necesarios para las figuras geométricas.

3.2.1. Muestra del Dataset MNIST

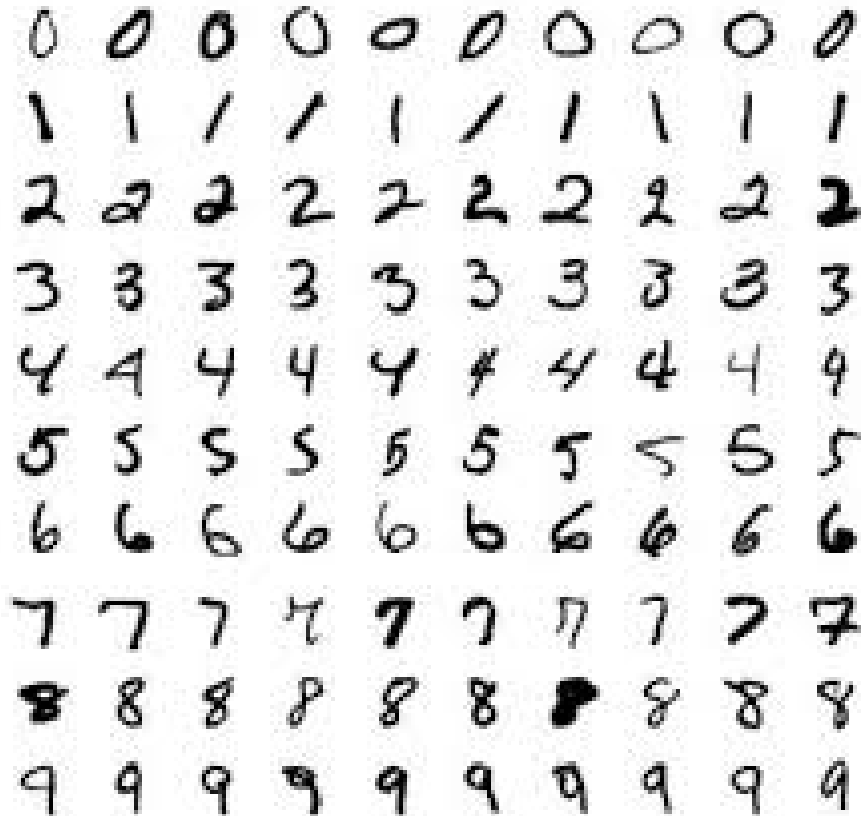


Figura 2: Muestra del Dataset MNIST

3.3. Diseño de Escenarios Experimentales

Para validar la hipótesis de investigación, se diseñaron tres escenarios controlados. En todos los casos, se reservó un 20 % del dataset total (aprox. 400 imágenes) como conjunto de prueba (*Test Set*) intocable, utilizado únicamente para la evaluación final.

1. Escenario 1: Escasez de Datos (Red C).

- **Condición:** Entrenamiento desde cero (*From Scratch*).
- **Datos de Entrenamiento:** Se utilizó únicamente el 30 % del subconjunto de entrenamiento disponible. Esto simula un entorno real donde la obtención de datos es costosa o limitada.

2. Escenario 2: Transfer Learning (Red A).

- **Condición:** Pre-entrenamiento en MNIST seguido de *Fine-Tuning*.
- **Proceso:** Primero se entrenó la red base con las 10 clases de MNIST. Posteriormente, se congelaron las capas convolucionales (extracción de características) y se reemplazó la capa densa de salida por una nueva capa de 8 neuronas. Finalmente, se re-entrenó la red con el **100 %** del dataset de figuras.

3. Escenario 3: Abundancia de Datos (Red B).

- **Condición:** Entrenamiento desde cero (*From Scratch*).
- **Datos de Entrenamiento:** Se utilizó el **100 %** del dataset de entrenamiento disponible. Este escenario representa el “ideal” teórico y sirve como línea base superior (*Upper Bound*).

3.4. Arquitectura de la Red Neuronal

Para asegurar una comparación justa, los tres modelos comparten la misma arquitectura base secuencial, variando únicamente en la inicialización de sus pesos. La arquitectura consta de 241,288 parámetros entrenables (en el caso de entrenamiento desde cero) y se detalla a continuación:

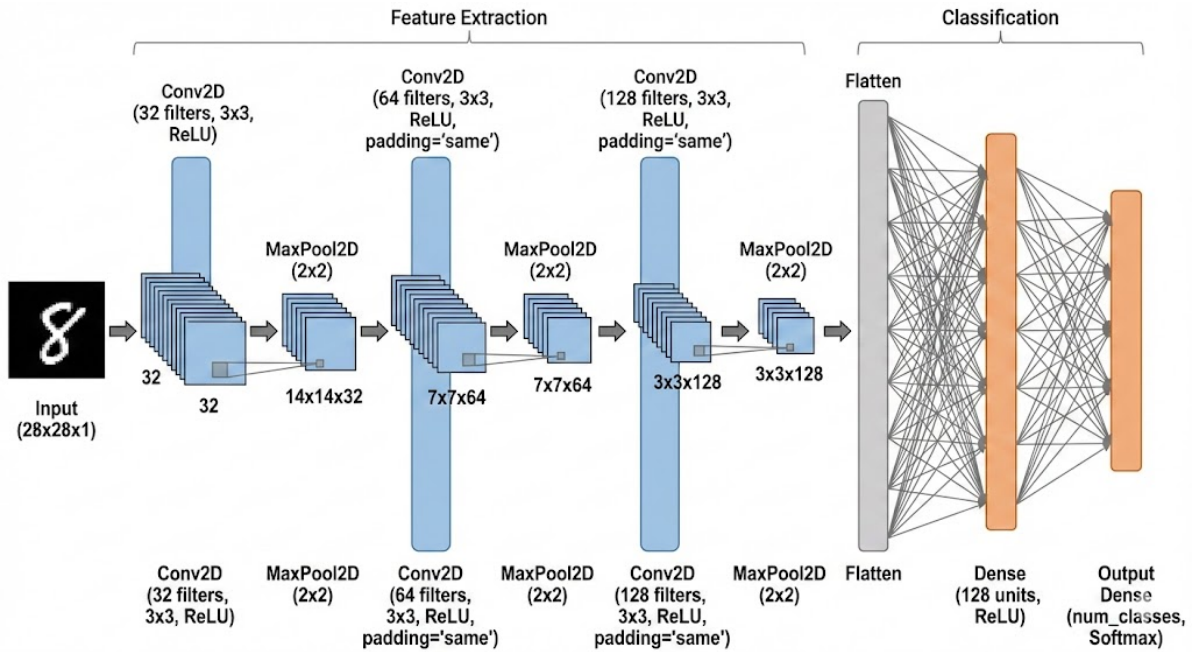


Figura 3: Diagrama esquemático de la arquitectura CNN utilizada.

A continuación, se detalla la estructura capa por capa, seguida de su implementación directa en código:

- **Entrada:** Tensor de forma $(28, 28, 1)$ (Imágenes en escala de grises).
- **Bloque Convolutivo 1:** Conv2D (32 filtros, kernel 3×3 , ReLU) + MaxPooling2D (2×2).
- **Bloque Convolutivo 2:** Conv2D (64 filtros, kernel 3×3 , ReLU, padding='same') + MaxPooling2D (2×2).

- **Bloque Convolutivo 3:** Conv2D (128 filtros, kernel 3×3 , ReLU, padding='same') + MaxPooling2D (2×2).
- **Aplanado:** Capa Flatten (convierte mapas de características 3D a vector 1D).
- **Capas Densas:**
 - Dense (128 neuronas, activación ReLU).
 - **Salida:** Dense (N clases, activación Softmax), donde $N = 8$ para el experimento principal.

La implementación en Keras/TensorFlow utilizada en el cuaderno Jupyter es la siguiente:

```

1 def build_model_from_reference(num_classes, input_shape=(28, 28, 1)):
2     """
3     Construye el modelo secuencial base utilizado para
4     la Red A, Red B y Red C.
5     """
6     model = Sequential([
7         # --- Extraccion de Caracteristicas ("Ojos") ---
8         # Bloque 1
9         Conv2D(32, 3, activation='relu', input_shape=input_shape),
10        MaxPool2D(),
11
12        # Bloque 2
13        Conv2D(64, 3, activation='relu', padding='same'),
14        MaxPool2D(),
15
16        # Bloque 3
17        Conv2D(128, 3, activation='relu', padding='same'),
18        MaxPool2D(),
19
20        # --- Clasificacion ("Cerebro") ---
21        Flatten(),
22        Dense(128, activation='relu'),
23        # Capa de Salida
24        Dense(num_classes, activation='softmax')
25    ])
26    return model

```

Listing 1: Implementación de la arquitectura CNN en Python

El entrenamiento de esta arquitectura se configuró utilizando el optimizador **Adam** [5] con una tasa de aprendizaje (*learning rate*) de 0,0001 y la función de pérdida *Sparse Categorical Crossentropy*. Cada escenario se ejecutó durante un total de 50 épocas con un tamaño de lote (*batch size*) de 16, monitoreando la métrica de *accuracy* en el conjunto de validación.

3.5. Detalles de Implementación

Dada la naturaleza experimental del estudio, se presentan a continuación los fragmentos de código críticos utilizados en el entorno Jupyter Notebook para definir la arquitectura y las estrategias de entrenamiento.

3.5.1. Definición de la Arquitectura Base

Se implementó una función constructora para instanciar modelos idénticos, garantizando que las diferencias en el rendimiento se deban exclusivamente a los datos y pesos iniciales, no a la topología de la red.

```
1 def build_model_from_reference(num_classes, input_shape=(28, 28, 1)):
2     model = Sequential([
3         # Bloque 1: Deteccion de bajo nivel
4         Conv2D(32, 3, activation='relu', input_shape=input_shape),
5         MaxPool2D(),
6         # Bloque 2: Caracteristicas medias
7         Conv2D(64, 3, activation='relu', padding='same'),
8         MaxPool2D(),
9         # Bloque 3: Caracteristicas complejas
10        Conv2D(128, 3, activation='relu', padding='same'),
11        MaxPool2D(),
12
13        Flatten(),
14        Dense(128, activation='relu'), # Capa densa intermedia
15        Dense(num_classes, activation='softmax') # Clasificador final
16    ])
17    return model
```

Listing 2: Función constructora del modelo CNN

3.5.2. Configuración de Transfer Learning (Red A)

Para la Red A, se aplicó la técnica de *feature extraction* congelando las capas convolucionales pre-entrenadas con MNIST y permitiendo el reentrenamiento solo de las capas densas ('top').

```
1 # Iteramos sobre las capas del modelo base (MNIST)
2 for layer in model_A_base.layers[:-1]:
3     # Congelamos solo capas convolucionales y de pooling (los "ojos")
4     if isinstance(layer, (Conv2D, MaxPool2D)):
5         layer.trainable = False
6     else:
7         # Dejamos las capas densas entrenables para el ajuste fino
8         layer.trainable = True
9
10 # Anadimos la nueva capa de salida para las 8 formas geometricas
11 model_A.add(Dense(8, activation='softmax', name='output_formas'))
```

Listing 3: Lógica de congelamiento de capas para Transfer Learning

3.5.3. Simulación de Escasez (Red C)

La condición de escasez se simuló mediante una estratificación rigurosa, seleccionando solo el 30 % de los datos disponibles para entrenamiento.

```
1 X_train_30, _, y_train_30, _ = train_test_split(  
2     X_train_full, y_train_full,  
3     train_size=0.30, # Restriccion al 30% de datos  
4     random_state=42,  
5     stratify=y_train_full # Mantiene proporciones de clases  
6 )
```

Listing 4: División del dataset para simular escasez

4. Resultados Experimentales

4.1. Análisis del Proceso de Entrenamiento

El comportamiento de los modelos durante las 50 épocas de entrenamiento revela dinámicas de aprendizaje distintivas para cada escenario.

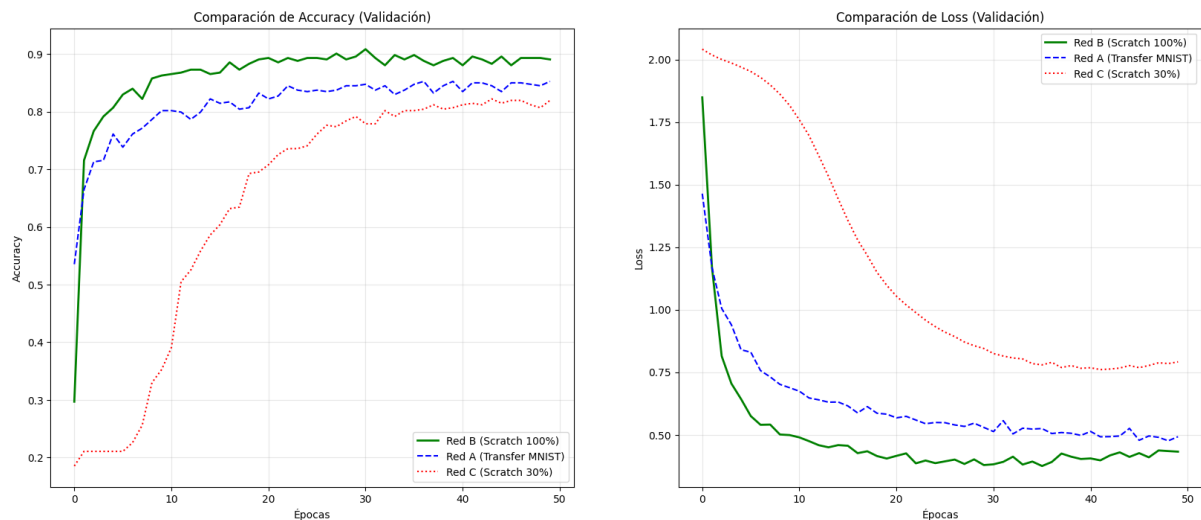


Figura 4: Curvas de Accuracy y Loss en el conjunto de validación para las tres redes.

Como se observa en la Figura 4:

- **Arranque en Frío (Red C - Línea Roja):** Inicia con una precisión muy baja ($< 25\%$) y muestra una curva de aprendizaje lenta e inestable. Su pérdida (*Loss*) final se mantiene alta (0.7919), indicando baja confianza en sus predicciones.
- **Ventaja de Transferencia (Red A - Línea Azul):** Gracias a los pesos de MNIST, la red inicia el entrenamiento con una precisión superior al 55 % desde la primera época, convergiendo mucho más rápido que la Red C.
- **Estabilidad (Red B - Línea Verde):** Muestra la curva más suave y ascendente, alcanzando rápidamente la asíntota de rendimiento máximo.

4.2. Evaluación Cuantitativa Final

Al finalizar el entrenamiento, los modelos fueron evaluados con el conjunto de prueba ($N = 394$ imágenes inéditas). Los resultados confirman la hipótesis de investigación ($C < A < B$).

Modelo	Accuracy Final	Loss Final	Observación
Red C (Escasez 30 %)	81.98 %	0.7919	Alto error de generalización.
Red A (Transfer Learning)	85.28 %	0.4929	Mejora significativa (+3.3 %).
Red B (Nativa 100 %)	89.09 %	0.4330	Mejor desempeño global.

Cuadro 1: Resumen comparativo de métricas finales.

4.3. Análisis Detallado por Clase

Para comprender las fortalezas de cada estrategia, analizamos las métricas de precisión y recall por figura geométrica. Se destacan dos hallazgos particulares en los reportes de clasificación:

1. **El Efecto “Curvas” (Semicírculo):** La Red A (Transfer) obtuvo una **Precisión del 96 %** en la clase “semicírculo”, superando notablemente a la Red B (72 %). Esto sugiere que los filtros aprendidos en MNIST (ricos en curvas de dígitos como 0, 3, 6, 8, 9) son excepcionalmente buenos para discriminar formas curvas, incluso mejor que el entrenamiento nativo.
2. **La Forma Difícil (Rombo):** Todas las redes sufrieron para clasificar el rombo, pero la Red C obtuvo el peor desempeño (F1-Score 0.71), confundiéndolo frecuentemente con cuadrados o triángulos debido a la falta de ejemplos suficientes para modelar sus ángulos específicos.

4.4. Matrices de Confusión

La visualización de errores permite identificar confusiones sistemáticas.

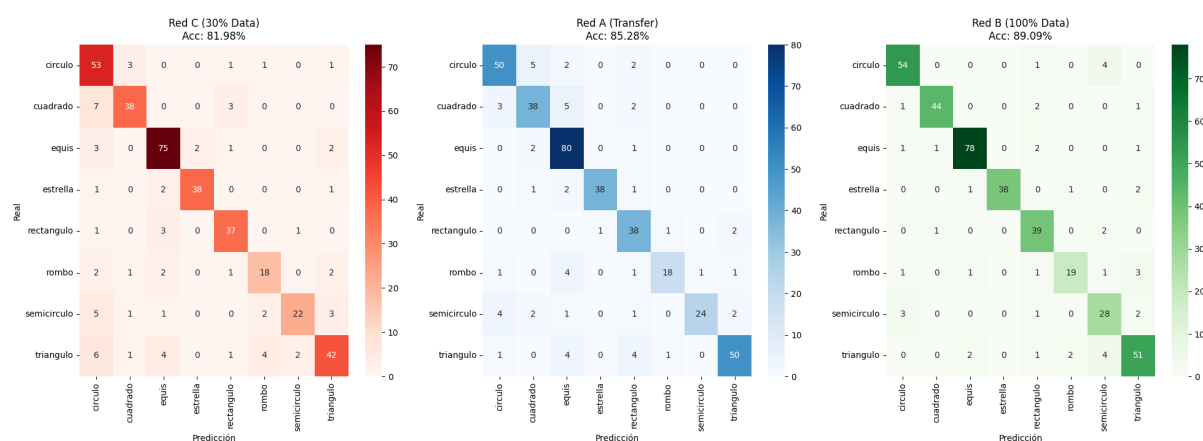


Figura 5: Matrices de confusión comparativas. Note la dispersión de errores fuera de la diagonal principal en la Red C (izquierda) frente a la diagonal limpia de la Red B (derecha).

Se observa claramente cómo la “mancha” de aciertos se concentra en la diagonal a medida que pasamos del escenario de escasez al de abundancia, con la Red A actuando como un puente intermedio efectivo.

5. Discusión y Análisis

El análisis de los resultados experimentales permite extraer interpretaciones profundas sobre la naturaleza del aprendizaje en redes neuronales bajo diferentes restricciones de información.

5.1. La Vulnerabilidad de la Escasez (Red C)

El desempeño inferior de la Red C (81.98 %) confirma la sensibilidad de las CNN a la falta de datos. Con solo el 30 % del dataset, la red no logró observar suficientes variaciones de cada forma geométrica. Esto se refleja en su alta función de pérdida final (0.7919), casi el doble que la de la Red B. Este fenómeno sugiere que, aunque la red lograba clasificar correctamente muchas imágenes, lo hacía con un margen de confianza muy bajo (*divinando con incertidumbre*). Las matrices de confusión revelaron que la Red C sufría de *confusión morfológica*, incapaz de distinguir consistentemente entre un cuadrado y un rectángulo, o un triángulo y un rombo, debido a que no aprendió a generalizar las proporciones de los ángulos y lados.

5.2. La Eficacia del Conocimiento Transferido (Red A)

El hallazgo más notable del estudio es la capacidad de la Red A para superar la barrera de la escasez, alcanzando un 85.28 % de exactitud. ¿Por qué filtros entrenados con números (MNIST) sirven para formas geométricas? La respuesta reside en las características visuales primitivas. Los dígitos manuscritos contienen curvas cerradas (0, 6, 8, 9) y líneas rectas (1, 4, 7). Un caso de éxito particular fue la clase **Semicírculo**. La Red A obtuvo una precisión del 96 % en esta clase, superando incluso a la Red B (nativa). Se teoriza que el pre-entrenamiento con dígitos curvos (especialmente el '3' y el '0') dotó a la red de filtros altamente especializados en detectar arcos, una ventaja que la Red B tuvo que aprender desde cero y que la Red C nunca logró consolidar.

5.3. La Supremacía de la Especialización (Red B)

A pesar de las ventajas del Transfer Learning, la Red B (100 % datos nativos) se mantuvo como el modelo superior (89.09 %). Esto valida que no existe sustituto perfecto para una base de datos robusta y específica del dominio. Mientras que la Red A adapta conocimientos ajenos, la Red B genera filtros optimizados exclusivamente para la morfología de las figuras geométricas objetivo, logrando la menor tasa de error y la mayor estabilidad durante el entrenamiento.

6. Conclusiones

Tras la ejecución de los tres escenarios experimentales y el análisis de las métricas resultantes, se presentan las siguientes conclusiones:

- **Validación de Hipótesis:** Se confirma la relación de rendimiento $C < A < B$. El Transfer Learning demostró ser una estrategia intermedia efectiva, mejorando el rendimiento en un **3.3 %** respecto al escenario de escasez.
- **Viabilidad ante la Escasez:** Se concluye que, en proyectos donde la recolección de datos es limitada (menos de 600 imágenes de entrenamiento), utilizar una red pre-entrenada en un dominio visualmente análogo (como MNIST para formas simples) es obligatorio para evitar el sobreajuste y mejorar la generalización.
- **Transferencia de Primitivas Visuales:** Se demostró que la transferencia de aprendizaje no requiere que las clases de origen y destino sean semánticamente iguales (números vs. formas), sino que compartan características geométricas de bajo nivel (trazos, bordes y curvas).
- **Limitación del Transfer Learning:** Aunque efectivo, el ajuste fino (*Fine-Tuning*) no logra igualar el rendimiento de un modelo entrenado con abundancia de datos nativos. Por tanto, el esfuerzo en la recolección y etiquetado de datos (DataOps) sigue siendo la inversión con mayor retorno en términos de precisión del modelo.

7. Referencias

- [1] Sinno Jialin Pan y Qiang Yang. «A survey on transfer learning». En: *IEEE Transactions on Knowledge and Data Engineering* 22.10 (2010), págs. 1345-1359.
- [2] Yann LeCun et al. «Gradient-based learning applied to document recognition». En: *Proceedings of the IEEE* 86.11 (1998), págs. 2278-2324.
- [3] Ian Goodfellow, Yoshua Bengio y Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [4] Jason Yosinski et al. «How transferable are features in deep neural networks?». En: *Advances in Neural Information Processing Systems*. Vol. 27. 2014.
- [5] Diederik P Kingma y Jimmy Ba. «Adam: A method for stochastic optimization». En: *arXiv preprint arXiv:1412.6980* (2014).